

DEPI GRADUATION CAPSTONE

RAG-Based Intelligent Support Ticket System

Automated ticket triage and grounded response drafting — combining a classical baseline, semantic retrieval, and an LLM generation layer with human fallback.

73.2%

Queue Prediction
Accuracy

90.3%

Retrieval
Performance

46.2K

Cleaned Tickets
Indexed

LIVE

Deployed on Azure
Container Apps

What we'll walk through

01

The Problem

Why support triage breaks at scale

02

System Architecture

From request to grounded response

03

Data Pipeline

Cleaning, filtering, embedding

04

Baseline Models

TF-IDF + SVM as the floor

05

RAG & Retrieval

FAISS, confidence, abstention

06

Deployment

Docker Compose → Azure

07

Results & Decisions

What worked, what we'd change

01 / OVERVIEW

The Problem

Manual triage doesn't scale with ticket volume

THE PROBLEM

From raw ticket to resolved issue

1

A user submits a ticket

Free-text description, no routing metadata attached.

2

Support staff read and classify

Every ticket needs a human to decide which queue it belongs to.

3

Similar past tickets go unused

Answered tickets sit in history instead of informing new replies.

4

Response time grows with volume

Backlogs form during peak periods; SLAs slip.

SYSTEM RESPONSE

Ticket embedded
(all-MiniLM-L6-v2)



Semantic search
over FAISS index



Queue predicted +
confidence scored



Reply drafted or
routed to a human

Technology stack

Data & ML

- pandas / NumPy
- spaCy lemmatization
- sentence-transformers
- scikit-learn (TF-IDF, SVM)

Retrieval & RAG

- FAISS (IndexFlatIP)
- Cross-encoder reranker
- Groq LLM API
- FastAPI

Frontend

- Next.js 15
- HeroUI components
- Server-side API proxy
- TypeScript

Infrastructure

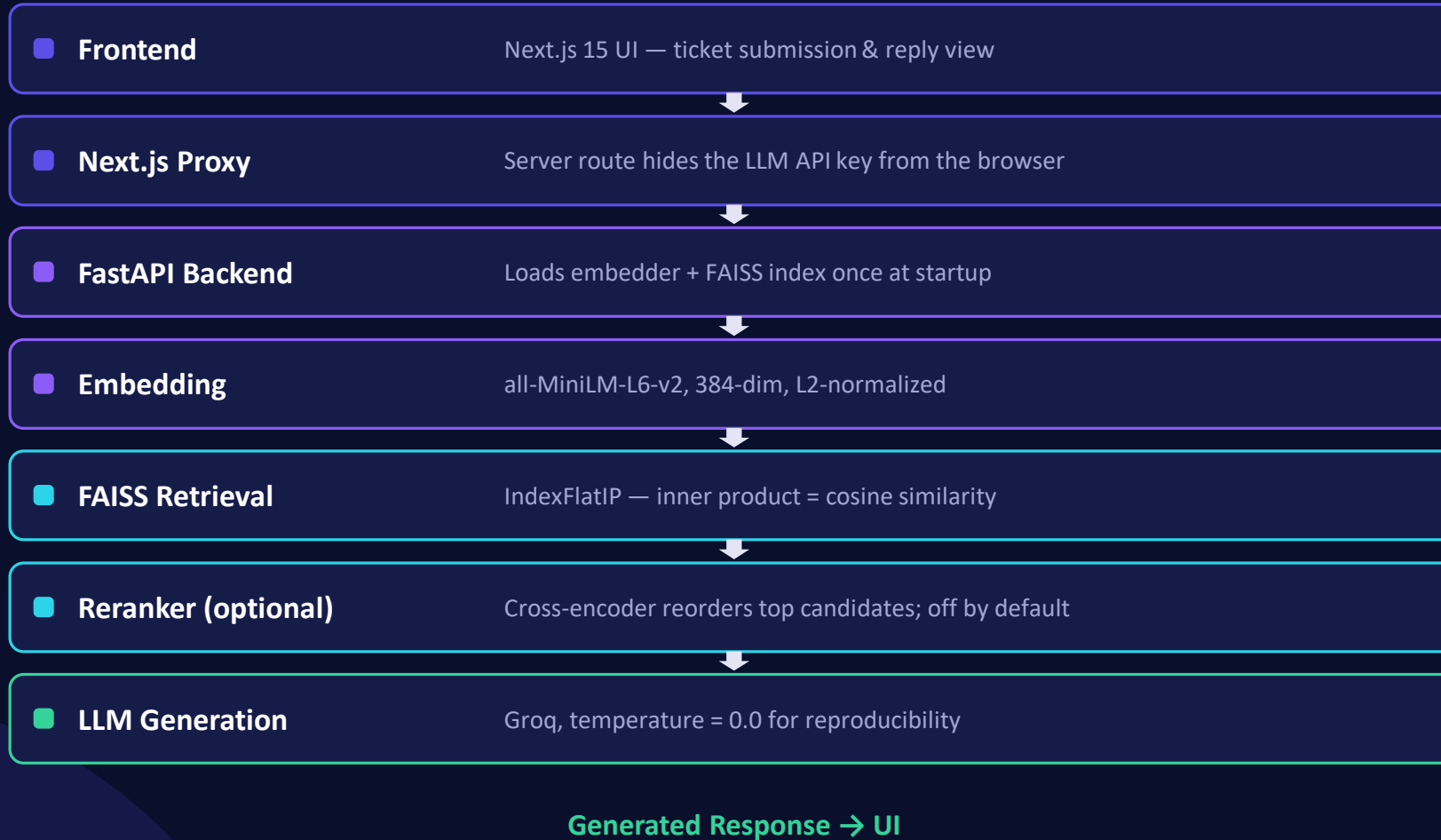
- Docker multi-stage builds
- Docker Compose (local)
- Azure Container Apps
- Azure Container Registry

02 / SYSTEM DESIGN

Architecture

From a browser request to a grounded, generated answer

One request, seven hops



Local Docker Compose → Azure Container Apps

LOCAL — DOCKER COMPOSE

api service

Port 8000 — health check, 45s start period

frontend service

Port 3000 — depends_on api: service_healthy

Env wiring

Backend URL passed to frontend via env var

Network

Bridge network triage-net links both services

PRODUCTION — AZURE CONTAINER APPS

Model baked at build time

Embedding model + FAISS index shipped inside the image

Min replicas = 1

Avoids cold starts on the embedder / index load

Build-time API URL

NEXT_PUBLIC_API_URL passed as a Docker build argument

Registry

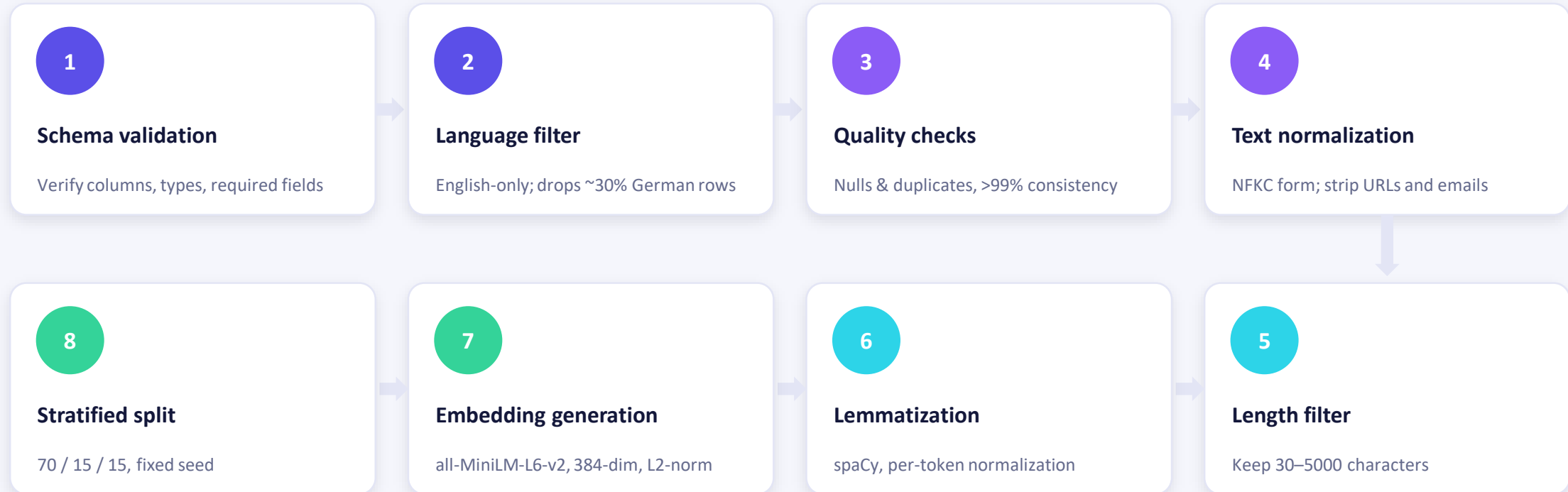
Images pushed to Azure Container Registry

03 / MODULE 1

Data Pipeline

Eight steps from raw export to model-ready splits

Raw export to model-ready splits



What the cleaned dataset looks like

ROW REDUCTION

Raw rows

61,800

Cleaned rows

46,200

~25% removed by language filtering, null/duplicate checks, and length bounds

>99%

Label Quality

~10x

Class Imbalance
(top vs smallest queue)

450 chars

Median ticket text length

220 tokens

95th percentile length

5–8%

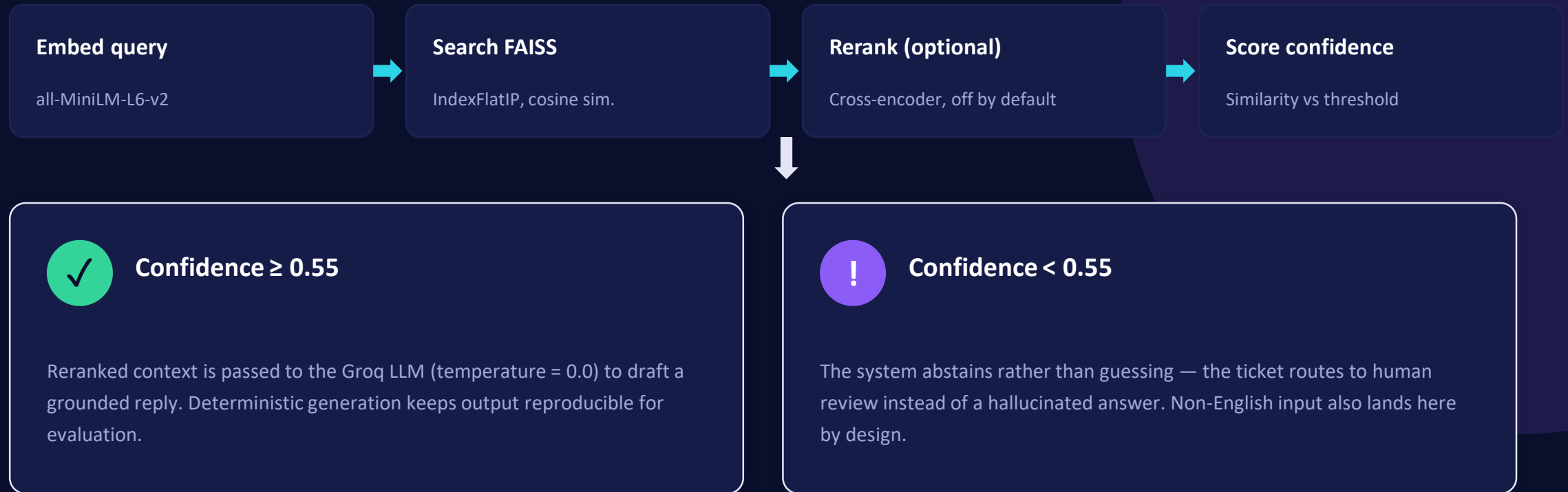
Duplicate template answers
(inflates BLEU — documented caveat)

04 / MODULE 2 & 3

Baseline, RAG & Retrieval

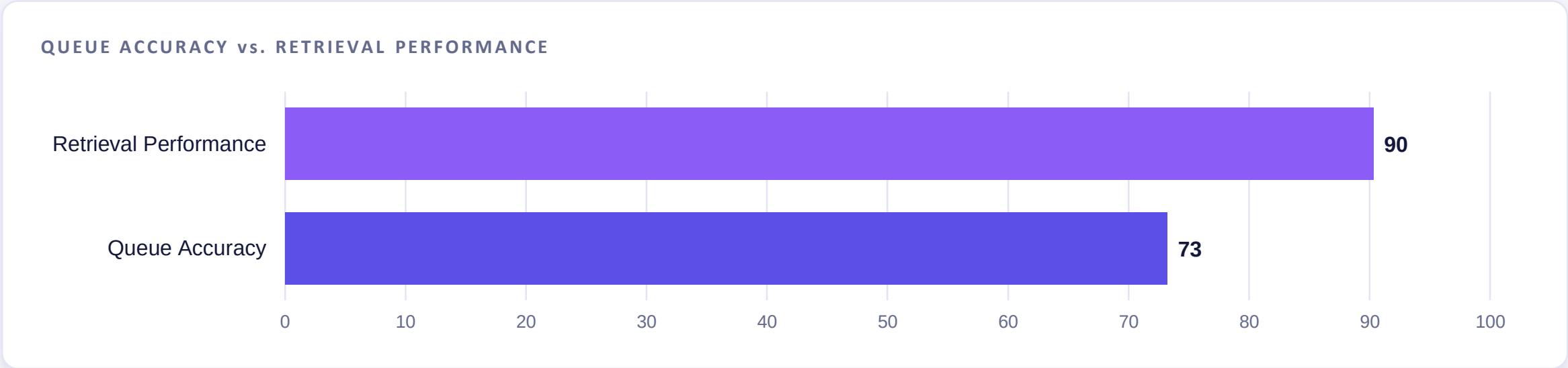
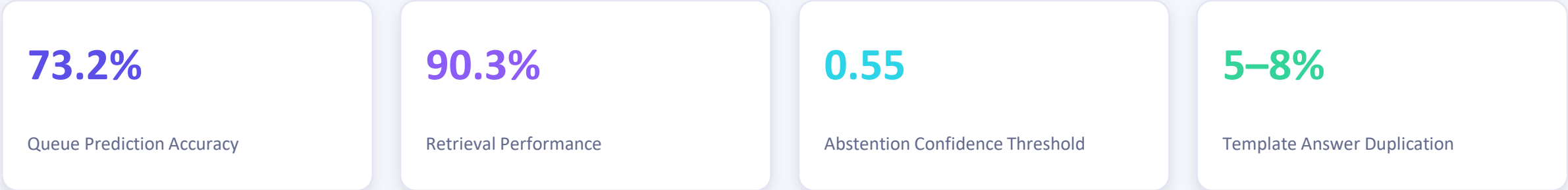
From a TF-IDF floor to confidence-gated generation

Retrieve, score, and decide before generating



RESULTS

Baseline vs. retrieval-augmented performance



Choices worth defending

1

Abstain before hallucinating

Low retrieval confidence routes to human review instead of guessing an answer.

2

Cosine via L2-normalized IP

FAISS IndexFlatIP on normalized vectors gives directly interpretable confidence scores.

3

Reranking decoupled from confidence

Cross-encoder reorders context for the prompt only — disabled by default to avoid extra latency.

4

API key isolation

A Next.js proxy route keeps the LLM key off the browser entirely.

5

Eager model loading

FastAPI lifespan loads the embedder and FAISS index once at startup, not per request.

6

Deterministic generation

temperature = 0.0 keeps classification and drafts reproducible for evaluation.

Thank you

RAG-Based Intelligent Support Ticket System — DEPI Graduation Capstone

- [Repo: github.com/nour-hatem/RAG-Based-Intelligent-Support-Ticket-System-DEPI-Project](https://github.com/nour-hatem/RAG-Based-Intelligent-Support-Ticket-System-DEPI-Project)
- [Live demo: triage-frontend.proudplant-cb7f23f0.eastus.azurecontainerapps.io](https://triage-frontend.proudplant-cb7f23f0.eastus.azurecontainerapps.io)

BUILT BY



Nour Hatem



Waad Sadek